

Тестовое задание Вариант №1

Описание программы

Данная программа осуществляет сортировку объектов по марке цвета. Допустимые значения которой (К, З, С). Порядок сортировки задается правилом по типу К>З>С. Для сортировки данных по маркам используются объекты с атрибутом mark, по типу класса Item.

class Item:

```
def __init__(self, mark, value):
```

```
    self.mark = mark
```

```
    self.value = value
```

Целью данного подхода является способность сортировать различные объекты, ведь в value может храниться число, строка, словарь или объект пользовательского типа данных. Также пользователь может добавлять атрибуты. В unit тестах (TC-UNIT-POS-06) показана возможность сортировки разнородных данных. В консольном приложении данные поступают в виде строки, каждая буква которой приводится к объекту Item (это выполняет функция parse_user_input из console_app.py) .

Варианты Запуска

- 1) Запуск консольного приложения - файл console_app.py (есть возможность вывода порядка при помощи флага show_order, пример в кейсе TC-CON-POS-09)
- 2) Запуск Unit Тестов - файл tests_unit.py.
- 3) Запуск теста производительности - файл performance_test.py
- 4) Для использования в своих проектах используйте файл sort_by_color_mark.py.

Условные обозначения в идентификаторах тестов

CON-POS	Позитивный кейс для консольного приложения
CON-ERR	Негативный кейс для консольного приложения
CON-EXIT	Выход из программы
CON-UI	Удобство пользователя
PERF	Производительность
UNIT-POS	Позитивный кейс для unit тестов
UNIT-ERR	Негативный кейс для unit тестов

Перечень Тест-кейсов для ручного тестирования

Позитивные тесты для консольного приложения

1.1 Название: Успешный ввод с первого раза (правило $K < 3 < C$)

1.2 ID: TC-CON-POS-01

1.3 Тип теста: Позитивный

1.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

1.5 Шаги

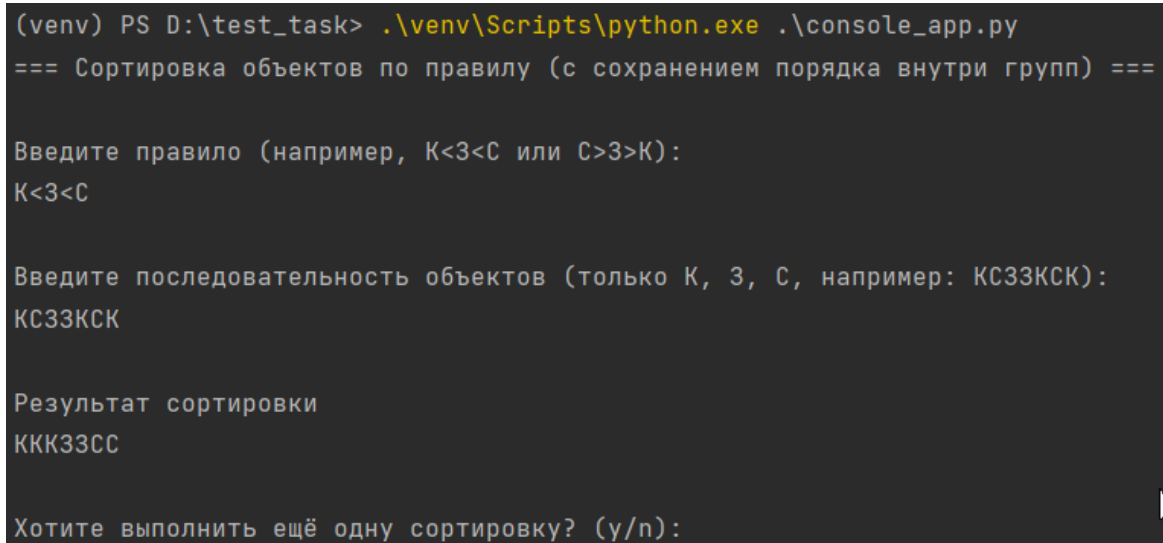
1.5.1 При запросе правила ввести $K < 3 < C$

1.5.2 При запросе последовательности ввести КСЗЗКСК

1.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

1.6 Ожидаемый результат

1.6.1 После шага 1.5.2:



```
(venv) PS D:\test_task> .\venv\Scripts\python.exe .\console_app.py
=== Сортировка объектов по правилу (с сохранением порядка внутри групп) ===

Введите правило (например, K<3<C или C>3>K):
K<3<C

Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):
КСЗЗКСК

Результат сортировки
КККЗЗСС

Хотите выполнить ещё одну сортировку? (y/n):
```

1.6.2 После шага 1.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

2.1 Название: Успешный ввод с первого раза (правило с '>' $C < 3 < K$, эквивалентно $K < 3 < C$)

2.2 ID: TC-CON-POS-02

2.3 Тип теста: Позитивный

2.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

2.5 Шаги

2.5.1 При запросе правила ввести C>3>K

2.5.2 При запросе последовательности ввести KC33KCK

2.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

2.6 Ожидаемый результат

2.6.1 После шага 2.5.2:

```
(venv) PS D:\test_task> .\venv\Scripts\python.exe .\console_app.py
=== Сортировка объектов по правилу (с сохранением порядка внутри групп) ===

Введите правило (например, K<3<C или C>3>K):
C>3>K

Введите последовательность объектов (только K, 3, C, например: KC33KCK):
KC33KCK

Результат сортировки
KKK33CC

Хотите выполнить ещё одну сортировку? (y/n):
```

2.6.2 После шага 2.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

3.1 Название: Правило с пробелами

3.2 ID: TC-CON-POS-03

3.3 Тип теста: Позитивный

3.4 Предварительное условие: Программа запущена. (python3 console_app.py).

3.5 Шаги

3.5.1 При запросе правила ввести K < 3 < C

3.5.2 При запросе последовательности ввести KC33KCK

3.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

3.6 Ожидаемый результат

3.6.1 После шага 3.5.2:

```
(venv) PS D:\test_task> .\venv\Scripts\python.exe .\console_app.py
=== Сортировка объектов по правилу (с сохранением порядка внутри групп) ===

Введите правило (например, K<3<C или C>3>K):
K < 3 < C

Введите последовательность объектов (только K, 3, C, например: KC33KCK):
KC33KCK

Результат сортировки
KKK33CC

Хотите выполнить ещё одну сортировку? (y/n):
```

3.6.2 После шага 3.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

4.1 Название: Последовательность без цвета 3

4.2 ID: TC-CON-POS-04

4.3 Тип теста: Позитивный

4.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

4.5 Шаги

4.5.1 При запросе правила ввести K<3<C

4.5.2 При запросе последовательности ввести KCKC

4.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

4.6 Ожидаемый результат

4.6.1 После шага 4.5.2:

```
(venv) PS D:\test_task> .\venv\Scripts\python.exe .\console_app.py
=== Сортировка объектов по правилу (с сохранением порядка внутри групп) ===

Введите правило (например, K<3<C или C>3>K):
K<3<C

Введите последовательность объектов (только K, 3, C, например: KC33KCK):
KCKC

Результат сортировки
KKCC

Хотите выполнить ещё одну сортировку? (y/n):
```

4.6.2 После шага 4.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

5.1 Название: Один цвет в последовательности (все К)

5.2 ID: TC-CON-POS-05

5.3 Тип теста: Позитивный

5.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

5.5 Шаги

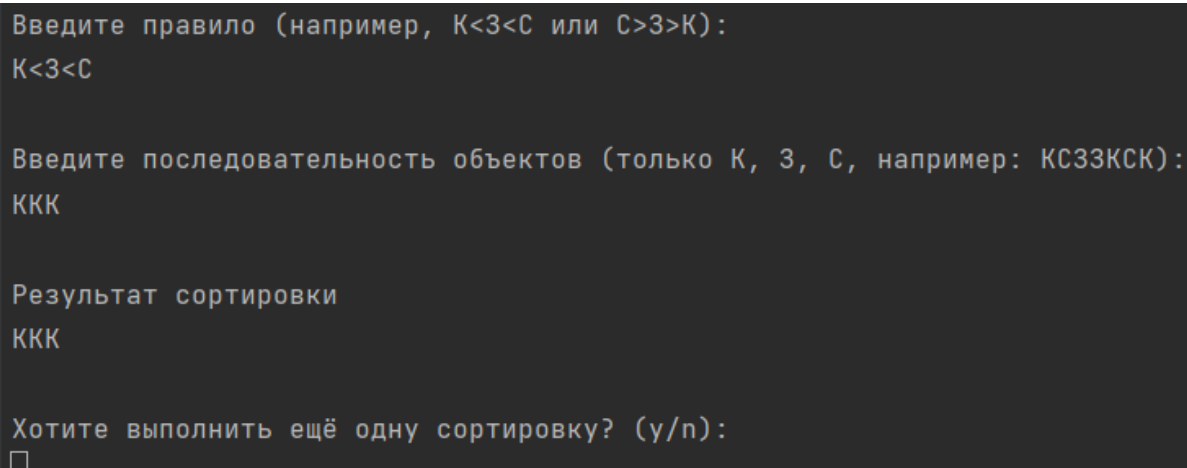
5.5.1 При запросе правила ввести $K < 3 < C$

5.5.2 При запросе последовательности ввести ККК

5.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

5.6 Ожидаемый результат

5.6.1 После шага 5.5.2:



```
Введите правило (например, K<3<C или C>3>K):  
K<3<C  
  
Введите последовательность объектов (только К, 3, С, например: KC33KCK):  
ККК  
  
Результат сортировки  
ККК  
  
Хотите выполнить ещё одну сортировку? (y/n):  
n
```

5.6.2 После шага 5.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

6.1 Название: Один символ в последовательности

6.2 ID: TC-CON-POS-06

6.3 Тип теста: Позитивный

6.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

6.5 Шаги

6.5.1 При запросе правила ввести $K < 3 < C$

6.5.2 При запросе последовательности ввести K

6.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

6.6 Ожидаемый результат

6.6.1 После шага 6.5.2:

```
Введите правило (например, K<3<C или C>3>K):
K<3<C

Введите последовательность объектов (только K, 3, C, например: KC33KCK):
K

Результат сортировки
K

Хотите выполнить ещё одну сортировку? (y/n):

```

6.6.2 После шага 6.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

7.1 Название: Последовательность из 1000 символов

7.2 ID: TC-CON-POS-07

7.3 Тип теста: Позитивный

7.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

7.5 Шаги

7.5.1 При запросе правила ввести K<3<C

7.5.2 Ввести длинную последовательность в 1000 символов, ее можно сгенерировать ".join(random.choice('K3C') for _ in range(1000)) .

7.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

7.6 Ожидаемый результат

Программа не вылетает при вводе 1000 символов, это главная цель проверки теста, проверка корректности сортировки 1000 символов лежит на тесте TC-PERF-01.

7.6.1 После шага 7.5.2:

9.1 Название: Проверка вывода порядка вывода последовательности (флаг show_order)

9.2 ID: TC-CON-POS-09

9.3 Тип теста: Позитивный

9.4 Предварительное условие: Программа запущена с флагом show_order. (**python3 console_app.py show_order**).

9.5 Шаги

9.5.1 Выполнить сортировку (например аналогично TC-CON-POS-01)

9.5.2 Обратить внимание на появление индексов в выводе, показывающих корректный порядок сортировки внутри групп.

9.5.3 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

9.6 Ожидаемый результат

9.6.2 После шага 9.5.2:

```
(venv) PS D:\test_task> .\venv\Scripts\python.exe .\console_app.py show_order
=== Сортировка объектов по правилу (с сохранением порядка внутри групп) ===

Введите правило (например, K<3<C или C>3>K):
K<3<C

Введите последовательность объектов (только K, 3, C, например: KC33KCK):
K3C3KCK

Исходная последовательность (индекс:метка):
0:K, 1:3, 2:C, 3:3, 4:K, 5:C, 6:K

Результат сортировки
В формате (метка(исходный индекс)):
K(0), K(4), K(6), 3(1), 3(3), C(2), C(5)

Кратко (только метки): KKK33CC

Порядок внутри каждой группы сохраняется – видно по возрастанию индексов.

Хотите выполнить ещё одну сортировку? (y/n):
```

9.6.2 После шага 9.5.3 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

Негативные тесты для консольного приложения

1.1 Название: Пустое правило → повторный ввод

1.2 ID: TC-CON-ERR-01

1.3 Тип теста: Негативный

1.4 Предварительное условие: Программа запущена. (О вариантах запуска смотреть README.md или раздел “варианты запуска”).

1.5 Шаги

1.5.1 При запросе правила нажать Enter

1.5.2 Увидеть сообщение "Ошибка: правило не может быть пустым"

1.5.3 Ввести корректное правило C<3<K

1.5.4 Ввести корректную последовательность KKCKЗСК

1.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

1.6 Ожидаемый результат

1.6.1 После шага 1.5.4:

```
(venv) PS D:\test_task> .\venv\Scripts\python.exe .\console_app.py
=== Сортировка объектов по правилу (с сохранением порядка внутри групп) ===

Введите правило (например, K<3<C или C>3>K):

Ошибка: правило не может быть пустым. Попробуйте снова.

Введите правило (например, K<3<C или C>3>K):
C<3<K

Введите последовательность объектов (только K, 3, C, например: KC3ЗKCK):
KKCKЗСК

Результат сортировки
ССЗКККК

Хотите выполнить ещё одну сортировку? (y/n):
```

1.6.2 После шага 1.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

2.1 Название: Правило с недопустимой меткой (X)

2.2 ID: TC-CON-ERR-02

2.3 Тип теста: Негативный

2.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

2.5 Шаги

2.5.1 При запросе правила ввести C<3<X

2.5.2 Увидеть сообщение " Ошибка в правиле: Недопустимая метка в правиле: 'X'. Допустимы: К, 3, С. Попробуйте снова. "

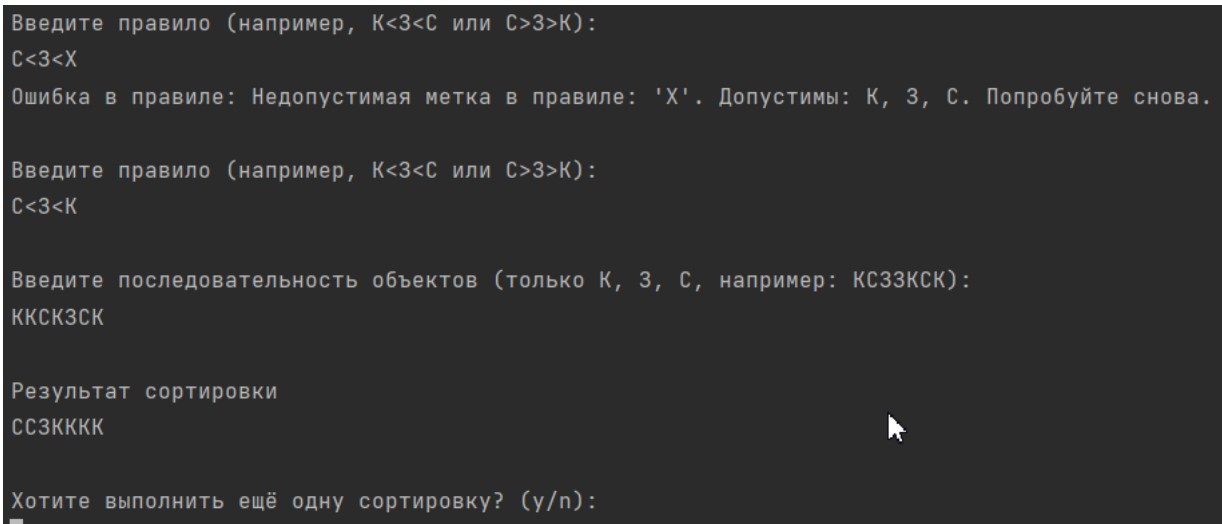
2.5.3 Ввести корректное правило C<3<К

2.5.4 Ввести корректную последовательность ККСКЗСК

2.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

2.6 Ожидаемый результат

2.6.1 После шага 2.5.4:



```
Введите правило (например, К<3<С или С>3>К):
C<3<X
Ошибка в правиле: Недопустимая метка в правиле: 'X'. Допустимы: К, 3, С. Попробуйте снова.

Введите правило (например, К<3<С или С>3>К):
C<3<К

Введите последовательность объектов (только К, 3, С, например: КСЗЗКСК):
ККСКЗСК

Результат сортировки
ССЗКККК

Хотите выполнить ещё одну сортировку? (y/n):
n
```

2.6.2 После шага 2.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

3.1 Название: Правило без разделителя (СЗК)

3.2 ID: TC-CON-ERR-03

3.3 Тип теста: Негативный

3.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

3.5 Шаги

3.5.1 При запросе правила ввести СЗК

3.5.2 Увидеть сообщение "Ошибка в правиле: Правило должно содержать разделители '<' или '>'. Попробуйте снова."

3.5.3 Ввести корректное правило C<3<K

3.5.4 Ввести корректную последовательность KKCKЗСК

3.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

3.6 Ожидаемый результат

3.6.1 После шага 3.5.4:

```
Введите правило (например, K<3<C или C>3>K):
C3K
Ошибка в правиле: Правило должно содержать разделители '<' или '>'. Попробуйте снова.

Введите правило (например, K<3<C или C>3>K):
C<3<K

Введите последовательность объектов (только K, 3, C, например: KC33KCK):
KKCKЗСК

Результат сортировки
CC3KKKK
```

3.6.2 После шага 3.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

4.1 Название: Правило с двумя цветами (K<3)

4.2 ID: TC-CON-ERR-04

4.3 Тип теста: Негативный

4.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

4.5 Шаги

4.5.1 При запросе правила ввести K<3

4.5.2 Увидеть сообщение "Ошибка в правиле: Правило должно содержать ровно 3 метки (K, 3, C), получено: ['K', '3']. Попробуйте снова."

4.5.3 Ввести корректное правило C<3<K

4.5.4 Ввести корректную последовательность KKCKЗСК

4.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

4.6 Ожидаемый результат

4.6.1 После шага 4.5.4:

```
Введите правило (например, K<3<C или C>3>K):
K<3
Ошибка в правиле: Правило должно содержать ровно 3 метки (K, 3, C), получено: ['K', '3']. Попробуйте снова.

Введите правило (например, K<3<C или C>3>K):
C<3<K

Введите последовательность объектов (только K, 3, C, например: KC33KCK):
KKCK3CK

Результат сортировки
CC3KKKK

Хотите выполнить ещё одну сортировку? (y/n):
```

4.6.2 После шага 4.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

5.1 Название: Правило с дубликатами (K<3<K)

5.2 ID: TC-CON-ERR-05

5.3 Тип теста: Негативный

5.4 Предварительное условие: Программа запущена. (**python3 console_app.py**)

5.5 Шаги

5.5.1 При запросе правила ввести K<3<K

5.5.2 Увидеть сообщение " Ошибка в правиле: Правило содержит повторяющиеся метки: ['K', '3', 'K']. Попробуйте снова."

5.5.3 Ввести корректное правило C<3<K

5.5.4 Ввести корректную последовательность KKCK3CK

5.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

5.6 Ожидаемый результат

5.6.1 После шага 5.5.4:

```
Введите правило (например, K<3<C или C>3>K):
K<3<K
Ошибка в правиле: Правило содержит повторяющиеся метки: ['K', '3', 'K']. Попробуйте снова.

Введите правило (например, K<3<C или C>3>K):
C<3<K

Введите последовательность объектов (только K, 3, C, например: KC33KCK):
KKCK3CK

Результат сортировки
CC3KKKK

Хотите выполнить ещё одну сортировку? (y/n):
```

5.6.2 После шага 5.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

6.1 Название: Смешанные разделители в правиле (K<3>C)

6.2 ID: TC-CON-ERR-06

6.3 Тип теста: Негативный

6.4 Предварительное условие: Программа запущена. (**python3 console_app.py**)

6.5 Шаги

6.5.1 При запросе правила ввести K<3>C

6.5.2 Увидеть сообщение "Ошибка в правиле: Правило не должно содержать одновременно '<' и '>'. Попробуйте снова."

6.5.3 Ввести корректное правило C<3<K

6.5.4 Ввести корректную последовательность KKCK3CK

6.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

6.6 Ожидаемый результат

6.6.1 После шага 6.5.4:

```
Введите правило (например, K<3<C или C>3>K):
K<3>C
Ошибка в правиле: Правило не должно содержать одновременно '<' и '>'. Попробуйте снова.

Введите правило (например, K<3<C или C>3>K):
C<3<K

Введите последовательность объектов (только K, 3, C, например: KC33KCK):
KKCK3CK

Результат сортировки
CC3KKKK

Хотите выполнить ещё одну сортировку? (y/n):
```

6.6.2 После шага 6.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

7.1 Название: Пустая последовательность

7.2 ID: TC-CON-ERR-07

7.3 Тип теста: Негативный

7.4 Предварительное условие: Программа запущена. (**python3 console_app.py**)

7.5 Шаги

7.5.1 Ввести корректное правило $C < Z < K$

7.5.2 Ввести пустую последовательность (нажать Enter)

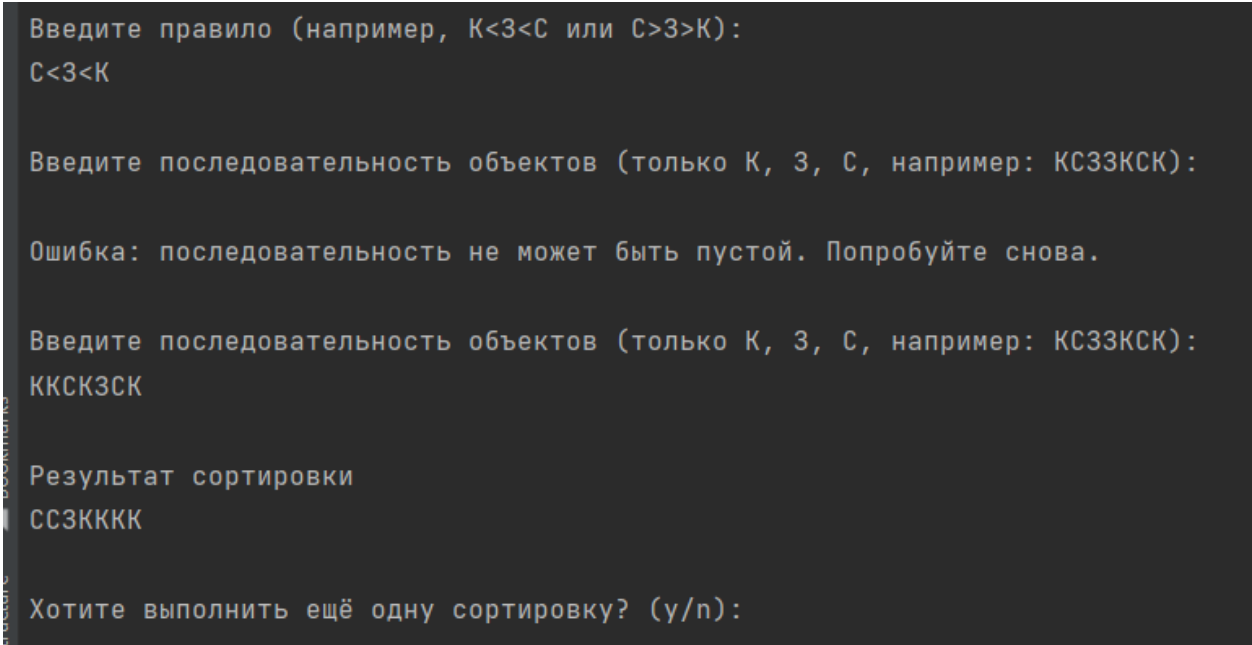
7.5.3 Увидеть сообщение " Ошибка: последовательность не может быть пустой. Попробуйте снова.

7.5.4 Ввести корректную последовательность KKCKZCK

7.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

7.6 Ожидаемый результат

7.6.1 После шага 7.5.4:



```
Введите правило (например, K<Z<C или C>Z>K):  
C<Z<K  
  
Введите последовательность объектов (только K, Z, C, например: KCZCKK):  
  
Ошибка: последовательность не может быть пустой. Попробуйте снова.  
  
Введите последовательность объектов (только K, Z, C, например: KCZCKK):  
KKCKZCK  
  
Результат сортировки  
CCZKKKK  
  
Хотите выполнить ещё одну сортировку? (y/n):
```

7.6.2 После шага 7.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

8.1 Название: Недопустимый символ в последовательности

8.2 ID: TC-CON-ERR-08

8.3 Тип теста: Негативный

8.4 Предварительное условие: Программа запущена. (**python3 console_app.py**)

8.5 Шаги

8.5.1 Ввести корректное правило $C < Z < K$

8.5.2 Ввести последовательность KССКЗХ

8.5.3 Увидеть сообщение "Ошибка: Недопустимый символ 'X'. Допустимы: К, З, С. Попробуйте снова."

8.5.4 Ввести корректную последовательность ККСКЗСК

8.5.5 На вопрос "Хотите выполнить ещё одну сортировку?" ввести n

8.6 Ожидаемый результат

8.6.1 После шага 8.5.4:

```
Введите правило (например, K<Z<C или C>Z>K):
C<Z<K

Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):
КССКЗХ
Ошибка: Недопустимый символ 'X'. Допустимы: К, З, С. Попробуйте снова.

Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):
ККСКЗСК

Результат сортировки
ССЗКККК

Хотите выполнить ещё одну сортировку? (y/n):
```

8.6.2 После шага 8.5.5 программа корректно завершает работу – видим надпись "Process finished with exit code 0"

Тесты завершения программы для консольного приложения

1.1 Название: Выход по любому символу кроме д/да/у/yes (без учета регистра)

1.2 ID: TC-CON-EXIT-01

1.3 Тип теста: Выход из программы

1.4 Предварительное условие: Программа запущена. (**python3 console_app.py**), выполнена успешная сортировка – как это сделать можно увидеть в тесте TC-CON-POS-01

1.5 Шаги

1.5.1 Увидеть сообщение " Хотите выполнить ещё одну сортировку? (y/n):

1.5.2 Ввести символы **не** из д/да/у/yes (без учета регистра)

1.6 Ожидаемый результат

1.6.1 После шага 1.5.2:

```
Хотите выполнить ещё одну сортировку? (y/n):  
123  
До свидания!  
  
Process finished with exit code 0
```

2.1 Название: Выход по прерыванию с клавиатуры

2.2 ID: TC-CON-EXIT-02

2.3 Тип теста: Выход из программы

2.4 Предварительное условие: Программа запущена. (**python3 console_app.py**)

2.5 Шаги

2.5.1 Выполнить прерывание с клавиатуры на разных этапах работы программы (запрос правила, запрос последовательности, запрос согласия на повтор).

2.6 Ожидаемый результат

2.6.1 После шага 2.5.1:

```
Введите правило (например, К<З<С или С>З>К):  
Программа прервана пользователем  
  
Process finished with exit code 0
```

Тест удобства использования для консольного приложения

1.1 Название: Проверка сообщения об ошибке раскладки клавиатуры

1.2 ID: TC-CON-UI-01

1.3 Тип теста: Удобство использования

1.4 Предварительное условие: Программа запущена. (**python3 console_app.py**).

1.5 Шаги

1.5.1 Проверить сообщение о приглашении к вводу правила “ Введите правило (например, К<З<С или С>З>К):”.

1.5.2 При запросе правила ввести К<З<С(с английской раскладки). Проверить сообщение “Ошибка в правиле: Недопустимая метка в правиле: 'С' Проверьте раскладку клавиатуры. Допустимы: К, З, С. Попробуйте снова.”.

1.5.3 Ввести правило К<З<С (на это раз на кириллице), проверить новое сообщение приглашение к вводу “ Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):”

1.5.4 Ввести последовательность К(с английской раскладки)ККЗСС. Увидеть сообщение “Ошибка: Недопустимый символ 'К' Проверьте раскладку клавиатуры. Допустимы: К, З, С. Попробуйте снова.”

1.5.5 Ввести корректную последовательность на кириллице КККЗСС

1.6 Ожидаемый результат

1.6.1 После шага 1.5.5:

```
Введите правило (например, К<З<С или С>З>К):
К<З<С
Ошибка в правиле: Недопустимая метка в правиле: 'С' Проверьте раскладку клавиатуры. Допустимы: К, З, С. Попробуйте снова.

Введите правило (например, К<З<С или С>З>К):
К<З<С

Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):
КККЗСС
Ошибка: Недопустимый символ 'К' Проверьте раскладку клавиатуры. Допустимы: К, З, С. Попробуйте снова.

Введите последовательность объектов (только К, З, С, например: КСЗЗКСК):
КККЗСС

Результат сортировки
КККЗСС

Хотите выполнить ещё одну сортировку? (y/n):
```

Тест производительности

1.1 Название: Проверка производительности (скорость и отсутствие утечек памяти)

1.2 ID: TC-PERF-01

1.3 Тип теста: Производительность

1.4 Предварительное условие: Запущены два терминала на linux системе.

1.5 Шаги

1.5.1 В терминале №1 запустить скрипт **python3 performance_test.py**

1.5.2 В терминале №1 появятся данные о сортировке 1000 элементов и затрате времени на это (ожидаем меньше 1с). Также появится идентификатор процесса, нужно скопировать его и использовать в команде `ps -p {pid} -o pid,rss,vsz` в терминале №2. Данные будут в формате `rss` – физическая память в Кб, `vsz` – виртуальная память в Кб:

```
lila@ubuntu:~/color_mark_porject/test_task_color_mark_sort$ ps -p 5346 -o pid,rss,vsz
  PID   RSS    VSZ
  5346 10656 18964
```

1.5.3 В терминале №1 нажать Enter, программа выполнит сортировку 1000 элементов 10 раз сравнивая на каждой итерации расход памяти при помощи `tracemalloc`.

1.5.4 Удостовериться что программа еще не завершена и повторить команду из шага 1.5.2.

1.6 Ожидаемый результат

Терминал №1

```
lila@ubuntu:~/color_mark_porject/test_task_color_mark_sort$ python3 performance_test.py
PID процесса: 2003 (для оценки памяти ps -p 2003 -o pid,rss,vsz)
Время выполнения: 0.0025 секунд
Функция выполнена.
Нажмите Enter, чтобы выполнить функцию повторно 10 раз и сравнить статистику использования памяти...

--- После итерации 1 ---
/usr/lib/python3.8/tracemalloc.py:397: size=576 B (+576 B), count=4 (+4), average=144 B
/usr/lib/python3.8/tracemalloc.py:534: size=472 B (+472 B), count=2 (+2), average=236 B
performance_test.py:44: size=424 B (+424 B), count=1 (+1), average=424 B
performance_test.py:39: size=0 B (-424 B), count=0 (-1)
/usr/lib/python3.8/random.py:200: size=356 B (+356 B), count=2 (+2), average=178 B
--- После итерации 2 ---
/usr/lib/python3.8/tracemalloc.py:209: size=848 B (+848 B), count=2 (+2), average=424 B
/usr/lib/python3.8/tracemalloc.py:165: size=816 B (+816 B), count=2 (+2), average=408 B
/usr/lib/python3.8/tracemalloc.py:472: size=744 B (+744 B), count=9 (+9), average=83 B
/usr/lib/python3.8/tracemalloc.py:519: size=568 B (+568 B), count=1 (+1), average=568 B
/usr/lib/python3.8/tracemalloc.py:131: size=548 B (+548 B), count=3 (+3), average=183 B
--- После итерации 3 ---
/usr/lib/python3.8/tracemalloc.py:125: size=1816 B (+1312 B), count=20 (+13), average=91 B
/usr/lib/python3.8/tracemalloc.py:185: size=1248 B (+864 B), count=26 (+18), average=48 B
/usr/lib/python3.8/tracemalloc.py:521: size=1264 B (+832 B), count=3 (+2), average=421 B
/usr/lib/python3.8/tracemalloc.py:472: size=1464 B (+720 B), count=27 (+18), average=54 B
/usr/lib/python3.8/tracemalloc.py:131: size=0 B (-548 B), count=0 (-3)
--- После итерации 4 ---
/usr/lib/python3.8/tracemalloc.py:125: size=72 B (-1744 B), count=1 (-19), average=72 B
/usr/lib/python3.8/tracemalloc.py:121: size=1800 B (+1296 B), count=25 (+18), average=72 B
/usr/lib/python3.8/tracemalloc.py:131: size=548 B (+548 B), count=3 (+3), average=183 B
/usr/lib/python3.8/tracemalloc.py:97: size=440 B (+440 B), count=1 (+1), average=440 B
/usr/lib/python3.8/tracemalloc.py:102: size=0 B (-440 B), count=0 (-1)
--- После итерации 5 ---
/usr/lib/python3.8/tracemalloc.py:125: size=216 B (+144 B), count=3 (+2), average=72 B
/usr/lib/python3.8/tracemalloc.py:185: size=1392 B (+96 B), count=29 (+2), average=48 B
/usr/lib/python3.8/tracemalloc.py:472: size=1584 B (+80 B), count=30 (+2), average=53 B
/usr/lib/python3.8/tracemalloc.py:122: size=56 B (-56 B), count=2 (-2), average=28 B
/usr/lib/python3.8/tracemalloc.py:476: size=308 B (+28 B), count=11 (+1), average=28 B
--- После итерации 6 ---
/usr/lib/python3.8/tracemalloc.py:121: size=2464 B (+664 B), count=29 (+4), average=85 B
/usr/lib/python3.8/tracemalloc.py:131: size=0 B (-548 B), count=0 (-3)
/usr/lib/python3.8/tracemalloc.py:102: size=440 B (+440 B), count=1 (+1), average=440 B
/usr/lib/python3.8/tracemalloc.py:97: size=0 B (-440 B), count=0 (-1)
/usr/lib/python3.8/tracemalloc.py:125: size=72 B (-144 B), count=1 (-2), average=72 B
--- После итерации 7 ---
```

Сортировка выполняется < 1с. Также видно, что нет линейного роста потребления памяти между итерациями, самые затратные вызовы системные.

Терминал №2

```
ilia@ubuntu:~/color_mark_porject/test_task_color_mark_sort$ ps -p 2003 -o pid,rss,vsz
PID    RSS    VSZ
2003 10456 18956
ilia@ubuntu:~/color_mark_porject/test_task_color_mark_sort$ ps -p 2003 -o pid,rss,vsz
PID    RSS    VSZ
2003 10788 19192
ilia@ubuntu:~/color_mark_porject/test_task_color_mark_sort$
```

Значения в терминале 2 должны быть схожи у разных вызовов, (отличия в пределах 10% нормально и связано с фрагментацией), это говорит об отсутствии утечек памяти.

Перечень UNIT Тест-кейсов

TC-UNIT-POS-01	Проверка всех 12 валидных правил (6 перестановок цветов с двумя вариантами знака сравнения).
TC-UNIT-POS-02	Правило с пробелами.
TC-UNIT-POS-03	Пустая последовательность при верном правиле, ожидаем возврат пустого списка.
TC-UNIT-POS-04	Отсутствует метка в последовательности
TC-UNIT-POS-05	Сохранение исходного порядка внутри группы
TC-UNIT-POS-06	Объекты с дополнительными полями и значениями разных типов (корректная сортировка при наличии дополнительных атрибутов)
TC-UNIT-POS-07	Сортировка строки
TC-UNIT-NEG-01	Пустое правило, ожидаем ошибку ValueError "не может быть пустым"
TC-UNIT-NEG-02	Пустое КЗС, ожидаем ошибку ValueError "должно содержать разделители"
TC-UNIT-NEG-03	Смешанные разделители К<3>С, ожидаем ошибку ValueError "Правило не должно содержать одновременно '<' и '>'"
TC-UNIT-NEG-04	Только две метки в правиле К<3, ожидаем ошибку ValueError "Правило должно содержать ровно 3 метки"
TC-UNIT-NEG-05	4 метки в правиле К<3<С<К, ожидаем ошибку ValueError "Правило должно содержать ровно 3 метки"
TC-UNIT-NEG-06	Повторяющаяся метка в правиле К<3<К, ожидаем ошибку ValueError "Правило содержит повторяющиеся метки"
TC-UNIT-NEG-07	Недопустимый символ в правиле К<3<Х, ожидаем ошибку ValueError "Недопустимая метка в правиле: 'Х'"
TC-UNIT-NEG-08	Недопустимый латинский символ в правиле схожий с валидным К<3<С(кириллица), ожидаем ошибку ValueError "Недопустимая метка в правиле: 'С' Проверьте раскладку клавиатуры."
TC-UNIT-NEG-09	Объект с недопустимой меткой 'Х', [Item('K', 1), Item('X', 2), Item('C', 3)] ожидаем ошибку ValueError "Недопустимая метка у объекта: 'Х'"
TC-UNIT-NEG-10	Несколько объектов с недопустимыми метками [Item('P', 1), Item('Q', 2)] ожидаем ошибку ValueError "Недопустимая метка у объекта: 'Р'"
TC-UNIT-NEG-11	Объект не имеет атрибута .mark, Ожидаем AttributeError

Оценка покрытия Unit тестами

Для оценки покрытия был использован модуль coverage, он не является стандартным и требует установки (*pip install coverage*)

Прогон тестов и формирование отчета:

```
(venv) PS D:\test_task> coverage run -m unittest discover
.....
-----
Ran 18 tests in 0.006s

OK
(venv) PS D:\test_task> coverage report
Name                               Stmts   Miss  Cover
-----
sort_by_color_mark.py              58      5    91%
tests_unit.py                     100      1    99%
-----
TOTAL                             158      6    96%
(venv) PS D:\test_task> coverage html
Wrote HTML report to htmlcov\index.html
(venv) PS D:\test_task>
```

В отчете видим причину не 100% покрытия

Coverage for **sort_by_color_mark.py**: 91%

58 statements 53 run 5 missing 0 excluded

« prev ^ index » next coverage.py v7.6.1, created at 2026-06-01 13:51 +0700

```
1 ALLOWED_MARKS = {'K', 'З', 'C'}
2
3
4 class Item:
5     def __init__(self, mark, value):
6         self.mark = mark # Атрибут для сортировки
7         self.value = value # Тут может быть любое значение: число, строка, пользовательский тип данных
8
9     def __repr__(self):
10         return f"{self.mark}{self.value}"
11
12     def __eq__(self, other):
13         return self.mark == other.mark and self.value == other.value
14
```

```
# Пример использования
if __name__ == "__main__":
    data = [
        Item('K', 1), Item('C', 2), Item('З', 3),
        Item('З', 4), Item('K', 5), Item('C', [6])
    ]
    rule = "K<З<C"
    print(ColorSort.reorder(data, rule))
    print(ColorSort.reorder('KCЗЗKC', rule))
```

Метод `__repr__` используется для вывода в консоль, а пример использования вызывается только если файл запущен как самостоятельная программа, то есть покрытие полное.